

Không biết là series này sẽ kéo dài được bao lâu. Nó phụ thuộc vào nhiều yếu tố , thời gian công việc, hứng thú của mình nữa he he he. Tóm lại là đang nghiêm cứu về **System verilog**, thôi thì tổng hợp lại những gì đã đọc , thực hành được bỏ lên đây. Trước là cho mình (quên thì có nơi xem lại) , sau là cho người (ai có duyên vô tình vào đây lượm lặt gì thì cũng tốt). Cảm ơn đã ghé thăm cái chuồng gà nhè !!!!!!!!!!!!!!!!!!!!!

Yêu cầu để đọc hiểu series này là phải có kiến thức cơ bản về:

- Verilog : vì cách học là so sánh giữa verilog (viết tắt V) và system Verilog (viết tắt SV) nên không biết V thì xem như tèo!!!!!!
- Simulation tool verilog (model sim , nc verilog ...), biết wave form là cái gì.
- Biết search google ^^.
- Biết english.
- Phải siêng năng chịu khó đọc xem tác giả viết cái méo gì.

Tài liệu nghiên cứu :

- **SystemVerilog For Design Second Edition** – by
Stuart Sutherland
Simon Davidmann
Peter Flake
Foreword by Phil Moorby

Cách học : Thông qua các ví dụ , so sánh tương quan giữa V (cái đã biết) và SV (cái đang học) để làm sáng tỏ vấn đề. Code , đều được chạy mô phỏng để đảm bảo độ tin cậy là tác giả không viết nhầm cuội.

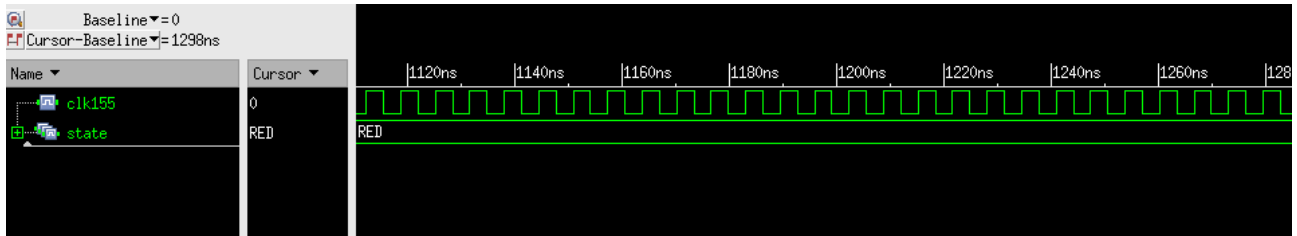
A. Mạch tuần tự

- Trong SV định nghĩa 3 loại `always_ff` (posedge clk), `always_latch` (posedge clk) thay thế cho `always` (posedge clk) trong V.
- `always_comb` trong SV thay thế cho `always(*)` trong V.
- kiểu enum được SV định nghĩa , giúp tường minh và tiết kiệm resource khi synthesis.
- Lưu ý SV chỉ là mở rộng của V nên những gì thuộc về V thì bạn cũng có thể code như vậy cho SV.

SystemVerilog enables modeling at a higher level of abstraction through the use of 2-state types, enumerated types, and userdefined types. These are complemented by new specialized always procedural blocks, `always_comb`, `always_ff` and `always_latch`. These and other new modeling constructs have been discussed in the previous chapters of this book.
(page 228)

Ví dụ 1 :

```
enum {RED , GREEN , YELLOW} state; // using enum defaults
always_ff @(posedge clk155)
begin
    state <= RED;
end
```

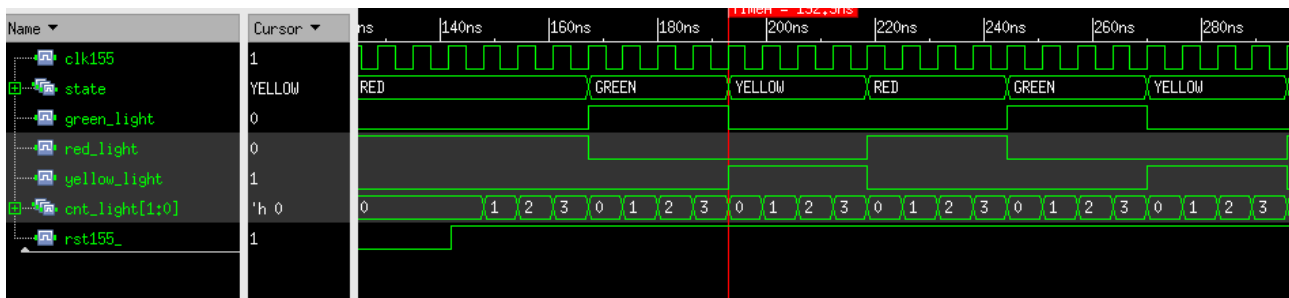


Ví dụ 2: có always_comb

```
enum {RED, GREEN, YELLOW} state;
logic [1:0] cnt_light;
always_ff @(posedge clk155)
begin
    case (state)
        RED : state <= &cnt_light ? GREEN : state;
        GREEN : state <= &cnt_light ? YELLOW : state;
        YELLOW : state <= &cnt_light ? RED : state;
        default : state <= RED;
    endcase

    cnt_light <= cnt_light + 1'b1;
end

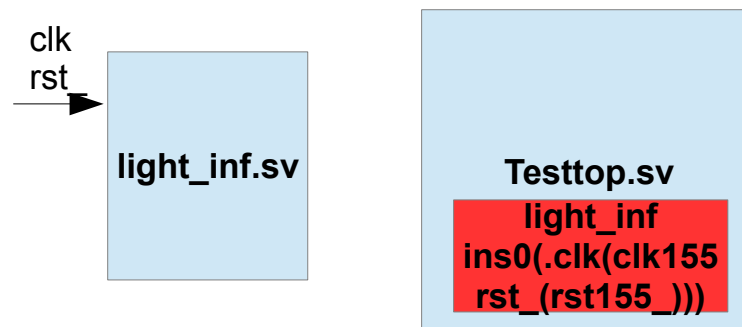
logic red_light; //logic 1-bit 4-state variable, like the Verilog reg type; can
// be declared as any vector size (discussed in Chapter 3).
logic green_light;
logic yellow_light;
always_comb
begin
    {green_light, yellow_light, red_light} = 3'b000;
    case (state)
        RED : red_light = 1'b1;
        GREEN : green_light = 1'b1;
        YELLOW : yellow_light = 1'b1;
    endcase
end
```



B. SV Interface : model interface bên ngoài module.

SV cung cấp 1 định nghĩa mới là interface. Interface có nhiều tác dụng giảm si sót khi instance các module, viết 1 module interface có thể sử dụng được ở nhiều nơi, ...

Ví dụ 1 : model interface <--> module



- Code 1 interface ex light_inf.v
- Code 1 module testtop.v
- Từ module testtop.v này có thể instance interface ligh_inf (cần phải import file light_inf.v *.flist)

code light_inf.v

```
interface light_if (input bit LIGHT_CLK, input bit LIGHT_RST_);
```

```
enum {RED, GREEN, YELLOW} state;
```

```
logic [1:0] cnt_light;
```

```
always_ff @(posedge LIGHT_CLK or negedge LIGHT_RST_)
```

```
begin
```

```
if (!LIGHT_RST_)
```

```
begin
```

```
cnt_light <= 2'd0;
```

```
state <= RED;
```

```
end
```

```
else
```

```
begin
```

```

case (state)
    RED : state <= &cnt_light ? GREEN : state;
    GREEN : state <= &cnt_light ? YELLOW : state;
    YELLOW : state <= &cnt_light ? RED : state;
    default : state <= RED;
endcase

cnt_light <= cnt_light + 1'b1;
end
end
endinterface : light_if

```

code testtop.sv

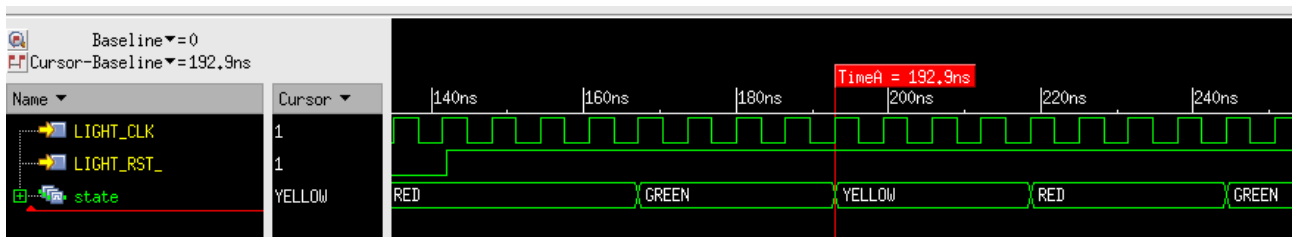
```

module testtop;

light_if light_if_test(.LIGHT_CLK (clk155),
                        .LIGHT_RST_(rst155_));

endmodule

```



Có thể instance interface ngay trong 1 khai báo input , output của 1 module hãy xem ví dụ bên dưới
 Ví dụ 2 : tham khảo và có hiệu chỉnh từ

[http://www.asic-world.com/systemverilog/interface1.html#Example : _Simple_Interface](http://www.asic-world.com/systemverilog/interface1.html#Example:_Simple_Interface)

code mem_if.sv

```

//+++++
// DUT With interface
//+++++
module simple_if (mem_if mif);
// Memory array
logic [7:0] mem [0:255];

//=====
// Read logic
//=====
always @ (posedge mif.clk_mif)
    if (mif.reset_mif) mif.datao_mif <= 0;
    else if (mif.ce_mif && ! mif.we_mif) mif.datao_mif <= mem[mif.addr_mif];

```

```
//=====
// Write Logic
//=====
always @ (posedge mif.clk_mif)
    if (mif.ce_mif && mif.we_mif) mem[mif.addr_mif] <= mif.datai_mif;
```

code simple_if.sv

```
//+++++
// DUT With interface
//+++++
module simple_if (mem_if mif);
// Memory array
logic [7:0] mem [0:255];

//=====
// Read logic
//=====
always @ (posedge mif.clk_mif)
    if (mif.reset_mif) mif.datao_mif <= 0;
    else if (mif.ce_mif && ! mif.we_mif) mif.datao_mif <= mem[mif.addr_mif];

//=====
// Write Logic
//=====
always @ (posedge mif.clk_mif)
    if (mif.ce_mif && mif.we_mif) mem[mif.addr_mif] <= mif.datai_mif;

endmodule
```

code testtop.sv

```
module testtop;
//=====
// Instianciate Interface and DUT
//=====
//mem_if miff(clk155);
//simple_if U_dut(miff);

logic reset_mif,ce_mif,we_mif;
logic [7:0] datai_mif,addr_mif;
wire [7:0] datao_mif;
//=====
// Instianciate Interface and DUT
//=====
mem_if miff(
    .clk_mif (clk155),
    .reset_mif (reset_mif),
```

```

        .ce_mif    (ce_mif),
        .we_mif    (we_mif),
        .datai_mif (datai_mif),
        .datao_mif (datao_mif),
        .addr_mif  (addr_mif)
    );

simple_if U_dut(.mif (miff));
//=====
// Test Vector generation
//=====
initial
begin
    reset_mif  <= 1;
    ce_mif     <= 1'b0;
    we_mif     <= 1'b0;
    addr_mif   <= 0;
    datai_mif  <= 0;
    repeat (10) @ (posedge clk155);
    reset_mif <= 0;
    for (int i = 0; i < 3; i ++ )
        begin
            @ (posedge clk155) ce_mif <= 1'b1;
            we_mif  <= 1'b1;
            addr_mif <= i;
            datai_mif <= $random;
            @ (posedge clk155) ce_mif <= 1'b0;
            $display ("@%0dns Write access address %x, data %x",
                $time,addr_mif, datai_mif);
        end
    for (int i = 0; i < 3; i ++ )
        begin
            @ (posedge clk155) ce_mif <= 1'b1;
            we_mif  <= 1'b0;
            addr_mif <= i;
            repeat (2) @ (posedge clk155);
            ce_mif  <= 1'b0;
            $display ("@%0dns Read access address %x, data %x",
                $time,addr_mif, datao_mif);
        end

    #10 $finish;

end

endmodule

```